ЛАБОРАТОРНАЯ РАБОТА № 2

Чтение информации из графических файлов

Цель работы

Изучить особенности хранения растровых изображений в форматах JPEG, GIF, TIFF, BMP, PNG и PCX. Освоить получение характеристик изображения путём самостоятельного чтения заголовков, маркеров, чанков и тегов.

Разработать веб-приложение для обработки отдельных файлов и папок с изображениями. Обеспечить отображение размеров, разрешения, глубины цвета и метода сжатия без использования специализированных библиотек получения метаданных.

Задачи работы

В ходе выполнения лабораторной работы были поставлены следующие задачи:

- изучить структуру графических форматов JPEG, GIF, TIFF, BMP, PNG и PCX;
- реализовать определение формата по сигнатуре;
- самостоятельно извлекать основные характеристики изображения;
- обеспечить выбор папки, содержащей до 100000 файлов;
- реализовать диапазонное чтение без загрузки крупных изображений целиком;
- организовать фоновую обработку с отображением прогресса;
- предусмотреть обнаружение типичных повреждений файлов;
- представить результаты в удобной таблице;
- разместить исходный код в GitHub и подготовить публикацию на Netlify.

Использованные средства разработки

- Rust — ручной разбор структуры графических файлов.
- WebAssembly — выполнение Rust-кода в браузере.
- wasm-bindgen и wasm-bindgen-futures — взаимодействие Rust с JavaScript и асинхронными операциями.

- js-sys — работа с JavaScript-функциями и массивами байтов.
- serde и serde_json — передача результатов в формате JSON.
- React и JavaScript — пользовательский интерфейс и управление обработкой.
- File API — доступ к выбранным файлам и чтение диапазонов.
- Web Workers — выполнение обработки вне основного потока интерфейса.
- Vite — локальный запуск и production-сборка.
- Playwright — подготовка браузерных тестов.
- Git и GitHub — контроль версий и размещение исходников.
- Netlify — настройка публикации веб-приложения.

Разработка и локальный запуск выполнялись на macOS. Библиотеки извлечения метаданных изображений не использовались. Подключённые зависимости обеспечивают взаимодействие с браузером и сериализацию, а сами форматы разбираются вручную.

Ход выполнения работы

1. Разработка интерфейса

Было создано веб-приложение Image Inspector на React и JavaScript.

В интерфейсе предусмотрены выбор папки или отдельных файлов, настройка количества фоновых обработчиков, остановка обработки и экспорт результатов.

Основная информация выводится в таблице:

- имя файла и относительный путь;
- фактический формат;
- ширина и высота в пикселях;
- разрешение по горизонтали и вертикали;
- глубина цвета;
- метод сжатия;
- статус обработки.

При нажатии на строку отображаются дополнительные характеристики, предупреждения, размер файла, объём прочитанных данных и время обработки. Добавлены поиск по имени и формату, а также фильтрация по статусу.

Для больших наборов используется виртуализация: в DOM создаются только видимые строки, а не вся таблица одновременно.

2. Выбор файлов и организация чтения

Для выбора файлов используется `input type="file"` с поддержкой множественного выбора. Выбор папки реализован через `showDirectoryPicker`, а при его недоступности — через файловый элемент с атрибутом `webkitdirectory`.

Обход учитывает вложенные каталоги. Сначала формируется список ссылок на файлы без чтения их содержимого. Установлено ограничение в 100000 файлов.

Чтение выполняется через:

javascript
```
file.slice(offset, end).arrayBuffer()
```

В Rust передаются размер файла и функция получения нужного диапазона байтов. Перед чтением проверяется, что смещение и длина не выходят за границы файла.

В JavaScript кэшируется область размером обычно 64 КиБ. Данные пикселей по возможности пропускаются по смещениям и длинам блоков. Поэтому крупное изображение не требуется загружать целиком для определения основных свойств.

3. Определение формата и разбор BMP, PNG

Формат определяется по сигнатуре, а не только по расширению. Например, для BMP проверяются байты `BM`, для PNG — полная восьмибайтная сигнатура, для JPEG — начальный маркер `FF D8`.

Если расширение не соответствует содержимому, программа сообщает об этом. Переименованный PNG определяется как PNG, а текстовый файл с расширением `.jpg` не принимается за изображение.

Для BMP вручную разбираются файловый заголовок и DIB-заголовок. Из них извлекаются геометрия, глубина, сжатие, плотность пикселей и сведения о палитре.

Поддерживаются `BI_RGB`, `BI_RLE8`, `BI_RLE4` и другие предусмотренные реализацией варианты. Палитра представляет собой таблицу цветов, а значение пикселя используется как индекс её элемента.

Для PNG выполняется последовательный разбор чанков. Из `IHDR` извлекаются ширина, высота, глубина отсчёта, тип цвета, методы сжатия и фильтрации, признак чересстрочности.

Глубина пикселя рассчитывается с учётом количества каналов. Например, RGB с 8 битами на компоненту соответствует 24 битам на пиксель, RGBA — 32 битам.

Чанк `pHYs` используется для определения плотности, `PLTE` — сведений о палитре, `IEND` — завершения структуры PNG.

4. Разбор JPEG и GIF

Для JPEG выполняется разбор маркеров и длин сегментов до начала первого сжатого скана.

Из SOF извлекаются размеры, точность компоненты и количество компонент. По разновидности SOF уточняется тип кодирования, например baseline или progressive.

Плотность определяется по JFIF и EXIF. EXIF содержит встроенную TIFF-структуру, поэтому для него применяется тот же принцип чтения тегов. При наличии подходящих значений EXIF они имеют приоритет над JFIF.

Для GIF разбираются логический экран, глобальная и локальные палитры, описания кадров и цепочки подблоков. В таблице выводится размер логического экрана и максимальная глубина индекса среди кадров.

Метод сжатия GIF обозначается как LZW. Абсолютный DPI не подставляется, поскольку отношение сторон пикселя не определяет физический размер изображения.

5. Разбор TIFF и PCX

Для TIFF определяется порядок байтов: little-endian или big-endian. Затем читаются каталоги IFD.

Каждая запись содержит идентификатор тега, тип, количество элементов и значение либо смещение. Если данные помещаются в четыре байта, они находятся непосредственно в записи; иначе считываются по указанному адресу.

Обрабатываются теги размеров, глубины компонент, сжатия, разрешения, палитры, смещений и размеров полос или плиток. Значения RATIONAL читаются как отношение числителя к знаменателю.

Для многостраничного TIFF просматривается цепочка IFD, а основные характеристики показываются для первой страницы.

Для PCX разбирается 128-байтный заголовок. Размер определяется по минимальным и максимальным координатам. Глубина рассчитывается как число бит на плоскость, умноженное на количество плоскостей.

Дополнительно проверяются способ кодирования, `BytesPerLine` и наличие палитры для соответствующего индексированного режима.

6. Обработка разрешения и глубины цвета

Разрешение по двум осям рассматривается отдельно. При хранении плотности в пикселях на метр используется формула:

text
```
DPI = pixelsPerMeter × 0,0254
```

Для пикселей на сантиметр применяется множитель 2,54.

Если физическая плотность отсутствует, выводится «—». Значения 72 или 96 DPI по умолчанию не подставляются.

Для палитрового изображения глубина означает число бит индекса, а не размер RGB-записи палитры. Для многокомпонентного изображения учитывается суммарная глубина компонент, включая альфа-канал при его наличии.

7. Подключение Rust и фоновая обработка

Для взаимодействия с браузером реализована асинхронная функция `inspect`. Она получает размер файла и функцию чтения диапазонов, выполняет разбор и возвращает результат в JSON.

Rust-модуль собирается командой:

bash
```
wasm-pack build ./rust --target web --release --out-dir
../public/wasm
```

Полученные JavaScript- и WASM-файлы размещаются среди статических ресурсов.

Обработка выполняется в пуле Web Workers. Каждый Worker имеет собственный экземпляр WASM и получает небольшие пакеты файлов. Освободившемуся обработчику назначается следующий пакет.

Главный поток принимает результаты и обновляет таблицу, прогресс и статистику. При остановке Workers завершаются, а готовые результаты сохраняются.

8. Проверка ошибок и размещение проекта

В парсерах проверяются длины полей, размеры блоков и допустимость смещений. Для PNG дополнительно проверяются служебные CRC и наличие `IEND`, для JPEG — наличие `EOI` в ограниченном хвостовом диапазоне. В TIFF обнаруживаются циклы цепочки IFD и выходы данных за границы.

При ошибке выводится статус «Файл повреждён» или «Не поддерживается». Ошибка одного файла не останавливает обработку остальных.

Подготовлены Rust unit-тесты и браузерные тесты Playwright. В контрольный набор включены шесть форматов, дополнительные варианты TIFF и JPEG, повреждённые файлы и изображения с изменёнными расширениями.

Исходники размещены в ветке `lab2` репозитория `the1nfm/color_lab`. Для Netlify настроена схема сборки из GitHub с каталогом публикации `dist`.

Проблемы, возникшие в работе, и способы решения

1. Ошибка линковки на macOS

Причина: не были приняты лицензионные соглашения Xcode.

Решение: выполнена команда `sudo xcodebuild -license`, после принятия соглашения сборка запущена повторно.

2. Неверный корень Git-репозитория

Причина: команды из папки приложения обращались к репозиторию домашнего каталога `/Users/matvei`.

Решение: корень проверен через `git rev-parse --show-toplevel`, после чего работа продолжена в правильной клонированной копии.

3. Предупреждение о вложенном репозитории

Причина: клонированный репозиторий оказался внутри исходной папки приложения и попал в список добавляемых файлов.

Решение: переход в нужную рабочую копию и исключение `.git`, вложенного репозитория и каталогов сборки при переносе исходников.

4. Ошибка `wasm-pack: not found` на Netlify

Причина: хостинг запускал `npm run build` без предварительной установки wasm-pack.

Решение: команда сборки дополнена установкой Rust toolchain, WebAssembly target и wasm-pack.

5. Различия в хранении характеристик

Причина: форматы используют разные единицы плотности и способы представления глубины цвета.

Решение: для каждого формата реализована отдельная логика интерпретации полей. Отсутствующий DPI отображается как «—», а глубина индекса не подменяется глубиной RGB-палитры.

Результаты работы

В результате выполнения лабораторной работы создано приложение Image Inspector, которое:

- вручную читает метаданные основных вариантов шести графических форматов;
- определяет формат по сигнатуре;
- выводит размеры, разрешение, глубину и сжатие;
- поддерживает обработку папки с ограничением до 100000 файлов;
- использует диапазонное чтение и Web Workers;
- показывает прогресс и позволяет остановить обработку;
- обнаруживает типичные структурные повреждения;
- поддерживает поиск, фильтрацию и экспорт CSV;
- предоставляет дополнительные пояснения характеристик.

Приложение собрано и запущено локально на macOS. Исходники размещены в отдельной ветке GitHub, подготовлена конфигурация публикации на Netlify.

Проверка является структурной: полное декодирование сжатых пикселей не выполняется. BigTIFF и отдельные нестандартные варианты форматов не поддерживаются. Численные результаты стресс-теста не приводятся, поскольку соответствующие измерения не зафиксированы.

Вывод

В ходе работы изучил структуру растровых форматов и реализовал самостоятельное получение характеристик изображения из массива байтов. На практике разобрал работу с сигнатурами, маркерами, чанками и TIFF-тегами.

Для парсинга использовал Rust, скомпилированный в WebAssembly, а для интерфейса — React. Диапазонное чтение позволило не загружать крупные изображения целиком, Web Workers — выполнять обработку вне основного потока, а виртуализация — сократить количество элементов таблицы.

Также выяснил, что разрешение и глубина требуют отдельной интерпретации для каждого формата. Практическим результатом стало веб-приложение для просмотра метаданных с ручными парсерами, фоновой обработкой и обнаружением типичных ошибок.